Requirements

SSON: The game shall be a fun, easy-to-understand, and straightforward bullet-hell game where the player has to survive an onslaught of geese and disperse them in various ways, while using different items and finding different events.

Selection of Requirements

Performance - The speed and quality of how the game operates.
Accessibility/Usability - The clarity and ease of use for any type of possible user.
Maintainability - The ability for the game to function correctly consistently.
Scalability - The game should be developed in a way such that it can be built upon easily.
Documentation - The amount/quality of documentation for deliverables that clarify how the system works.
Timing - The rate at which the game reacts to the user.
Operability - The game should be easy to understand and play immediately upon opening.
Reliability - The ability for the system to recover from glitches/failures.

These are the primary objectives that we require in this project, we have chosen our requirements such that they cover at least one (often many) of these objectives. These were chosen because they were the main points emphasised in our customer meeting, and we felt that they should be appropriately adapted and considered when engineering the requirements. Those in green we consider higher priority to those in yellow, but all objectives listed were considered during making the table of requirements.

The Tables of Requirements

Another important thing we considered was the readability and especially the traceability of the requirements; we believe that if they can't be easily referenced throughout the project they lose a lot of value, and people shouldn't be expected to search the whole list manually to find a specific requirement.

To help solve this, we split the requirements into 3 tables:

1. **User Requirements**:
   These are ordered (largely) alphabetically and then by priority, so that the most important user requirements are listed first, followed by the less important ones.
   ID - The relevant identifier for the requirement.
   Description - A description of the requirement in 1 or 2 sentences. Each requirement should adequately represent a need of the system.
   Priority - The word "shall" or "should" will be used in the description, and this represents the relative importance of the requirement to the system. The priority is an identifier for the key words for faster searching.

2. **Functional (System) Requirements**:
   These are ordered by user requirements, then priority. This allows the user to quickly search for a specific requirement just by looking at its location in the shorter user requirements list.
   ID, **Description**, **Priority** as above.
   Reference to User ID - Each functional requirement has a relevant user requirement that it is referencing, this is the system to connect FRs to their URs.
   Type - Shows if the requirement is a Transformation, Invariant or Failure.

3. **Quality (Non-Functional System) Requirements**:
   Also ordered by user requirements, then priority for the same reason as above.
   ID, **Description**, **Priority**, **Reference to User ID** as above.
   Type - Shows if a requirement is a Constraint (Con) or Quality (Qu).
   Fit Criterion/Test - The quantitative test needed to check that the requirement is being met.

User Requirements

ID	Description	Priority
UR_JAVA	The program shall run on Java 17.	Shall
UR_COMPUTE	The program shall be able to run fullscreen on a regular PC (Lab computer as a reference).	Shall
UR_START	The game shall open on a pause menu (to be started by the user).	Shall
UR_P_MOVE	The player shall be able to move independently.	Shall
UR_G_MOVE	The geese shall have movement AI to chase after the player.	Shall
UR_COUNTER	There shall be a counter of the number of items used/events interacted with.	Shall
UR_EVENTS	There shall be findable events throughout the map that have interactions with the player.	Shall
UR_INV	The player shall have access to an initial inventory of items used to deter/remove the geese.	Shall
UR_ITEM	The player shall be able to use the items in the inventory to distract the geese from player	Shall
UR_LIBRARY	The game shall use exclusively open-source libraries according to the relevant licence(s).	Shall
UR_MAP	There should be a map environment (resembling the UoY Campus) with appropriate collison and simple, clear layout.	Shall
UR_PAUSE	There shall be a button to allow a user to pause/unpause the game (and timer).	Shall
UR_RESULT	The player shall be told whether they won or lost when the round ends.	Shall
UR_SCORE	There shall be a score dependent on time remaining, number of items used, and events found.	Shall
UR_TIMER	There shall be a 5-minute timer visible counting down throughout one playthrough of the game.	Shall
UR_WINCOND	The game should end either after the 5-min timer expires or after the player is sufficiently “annoyed”, with both resulting in the game restarting.	Shall
UR_CLARITY	The gameworld should be visually easy and important objects (player, geese, items) should be obvious to all players with multiple mediums showing actions occurring.	Should
UR_CONTROL	There should be an easy-to-understand control scheme, both for movement, and collecting/using items.	Should
UR_EASY	The game should be simple to understand and play for an average ~16 year old student.	Should
UR_MAINTAIN	The game should be created in a way that it is easy to maintain/update.	Should
UR_NOHARM	The game should have an appropriate theme, tone and gameplay, with no actual harming of the geese or player.	Should
UR_VIEW	The game should be top-down (even in buildings).	Should

Functional Requirements

ID	Description	Ref. to User ID	Type	Priority
FR_START PAUSE	Every round (including when opening) shall start in the paused state, with the timer stopped at 05:00 until the player resumes.	UR_START, UR_PAUSE	Transformation	Shall
FR_MOVES PEED	The character and goose movement speed shall be constant.	UR_P_MOVE, UR_G_MOVE	Invariant	Shall
FR_MOVEK EYS	The system shall move the character when the arrow keys or WASD are pressed.	UR_P_MOVE	Transformation	Shall
FR_GEESE	The system shall make the geese come towards the player.	UR_G_MOVE	Transformation	Shall
FR_COUNT ERS	The system shall count and show the number of items used and events triggered.	UR_COUNTER	Transformation	Shall
FR_EVENT HIDDEN	The system shall place 2 hidden events on the map, invisible until the player reaches them.	UR_EVENTS	Transformation	Shall
FR_EVENT EFFECT	The system shall give an obvious positive or negative effect when an event is triggered.	UR_EVENTS, UR_CLARITY	Transformation	Shall
FR_EVENT ONCE	Nothing shall happen when the player walks over an event that has already been triggered.	UR_EVENTS	Failure	Shall
FR_EVENT TIME	Hidden events shall never change the remaining time.	UR_EVENTS, UR_TIMER	Invariant	Shall
FR_INVINT	The system shall give the player two initial items when the round starts.	UR_INV, UR_ITEM	Transformation	Shall
FR_ITEMC OUNT	The number of items in the inventory shall never be below 0.	UR_INV, UR_ITEM	Invariant	Shall
FR_ITEMU SE	The system shall remove an item from the inventory when it is used.	UR_ITEM	Transformation	Shall
FR_ITEMEF FECT	The system shall apply the effect of a used item to the geese in its range; the geese shall clearly react within 0.5 seconds.	UR_ITEM, UR_CLARITY	Transformation	Shall
FR_ITEMVA LID	The game state shall stay the same when the player tries to use an item not in the inventory.	UR_ITEM	Failure	Shall
FR_BOUND ARY	The system shall never place the character or geese outside the clearly marked map boundary.	UR_MAP, UR_VIEW, UR_CLARITY	Invariant	Shall
FR_PAUSE KEY	The system shall toggle between paused and playing when ESC is pressed.	UR_PAUSE	Transformation	Shall
FR_PAUSEI NPUT	The system shall ignore movement and item use while the game is paused.	UR_PAUSE	Invariant	Shall
FR_PAUSE FREEZE	The system shall stop the timer and the goose movement (within 100 ms) after pausing.	UR_PAUSE, UR_TIMER	Transformation	Shall
FR_WIN	The system shall win the round (and game)	UR_RESULT,	Transformation	Shall

	when at least all geese are dispersed.	UR_WINCOND		
FR_RESULTSCREEN	The system shall show the Win or Lose screen within 1 second after the round ends.	UR_RESULT, UR_WINCOND	Transformation	Shall
FR_SCORE	The game shall produce a quantifiable score at the end based on the time, items used and number of events found.	UR_SCORE	Transformation	Shall
FR_TIMEDISPLAY	The system shall display the remaining time (up to 300 seconds) on the screen at all times, within 1 second of the real play time.	UR_TIMER	Transformation	Shall
FR_TIMEOUT	The system loses the round when the timer reaches 0 and the player has not won.	UR_TIMER, UR_WINCOND	Transformation	Shall
FR_CONTROLS	The game shall be easily playable and use a WASD movement and single-button item use control scheme.	UR_CONTROL, UR_TIMER	Transformation	Shall
FR_HARMLESS	Items shall only attract, distract or drive away the geese, without any damage to them.	UR_NOHARM, UR_ITEM	Transformation	Shall
FR_NOWEAPON	The game shall have no item or event that injures or kills geese.	UR_NOHARM	Invariant	Shall
FR_VISUALS	The game shall have an aesthetically-striking, visually-clear top-down view of all sprites, item-uses, events and the map.	UR_VIEW, UR_CLARITY	Invariant	Shall

Quality Requirements

ID	Description	Ref. to User ID	Type	Fit criterion/test	Priority
QR_FPS	The system should run at an average of at least 60 FPS on a standard PC.	UR_EASY	Qu	Run several games and measure the average frame rate; it should be at least 60 FPS.	Should
QR_LEARN	The game should be easy to learn without instructions.	UR_EASY	Qu	Let 3 first-time players try the game; each should move and use an item within 2 minutes without help.	Should
QR_JAR	The system shall be delivered as an executable JAR with all dependencies.	UR_JAVA	Con	Double-click the JAR on a clean computer with Java 17; the game runs.	Shall
QR_LICENCE	The system shall only use libraries and assets whose licences allow our use. It should use no paid dependencies.	UR_LIBRARY	Con	List every library and asset with its licence; check each licence allows our use.	Shall
QR_HW	The system should run on a standard PC without a dedicated graphics card.	UR_COMPUTE	Con	Run the game on a university lab PC; check it runs smoothly.	Should
QR_KISS	The system shall not include any feature beyond the brief unless the team can explain why it has been implemented.	UR_SIMPLE, UR_MAINTAIN	Con	Review: every feature links to one UR; remove/update the ones that do not.	Shall